

Mikroelektronische Schaltungen und Systeme

Vorlesung 16 Memory

Prof. Jürgen Becker
Institut für Technik der Informationsverarbeitung (ITIV)



1. Memory Array
2. Random Access Memory
3. Read Only Memory
4. Lookup Tables



Memory Arrays

1

Memory Arrays

Einführung

- **Digitale Systeme** benötigen Speicher, um die vom System verwendeten Daten zu laden (**load**) und die vom System erzeugten Daten zu speichern (**store**).
- **Register**, die aus Flipflops bestehen, sind ein **hardwareintensiver** Speicher, der nur für kleine Datenmengen geeignet ist.
- In diesem Abschnitt werden drei **Arten von Speicher** vorgestellt:
 - **Dynamic Random Access Memory (DRAM)**
 - **Static Random Access Memory (SRAM)**
 - **Read Only Memory (ROM)**

Referenz S. 264 [1]

Memory Arrays

Einführung

- Speicher unterscheiden sich darin, wie er Bits speichern:
 - **Volatile**
 - Verliert seine Daten, wenn die Stromversorgung ausgeschaltet wird.
 - Vertreter:
 - **Random Access Memory (RAM)**
 - Auf jedes Datenwort kann mit demselben Delay zugegriffen werden. (Im Gegensatz zu Sequential Access Memory)
 - **Nonvolatile**
 - Behält seine Daten auch ohne Stromquelle.
 - Vertreter :
 - **Read Only Memory (ROM)**
 - Beachte, dass auch ROMs random accessible sind.
 - Beachte, dass moderne ROMs **auch schreibbar** sind (PROM, EEPROM, Flash-Speicher usw.).

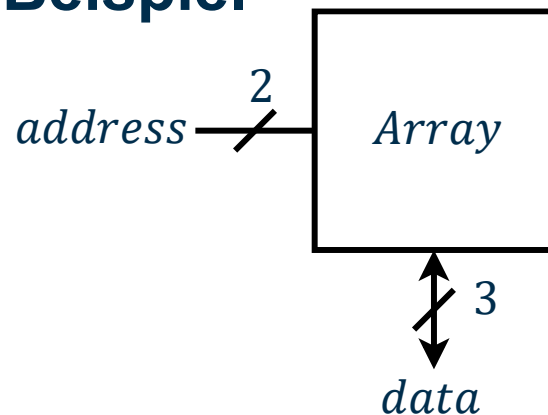
S. 264 [1]

Memory Arrays

Übersicht

- Memory ist als **zweidimensionales Array** von Zellen organisiert.
- Das System liest oder schreibt jeweils den Inhalt einer **Zeile des Arrays**. Die Zeile wird als **word** bezeichnet. Der **address input** gibt die Zeile an, die bearbeitet werden soll.
- **Depth** ist die Anzahl der Zeilen. **Width** ist die Anzahl der Spalten.

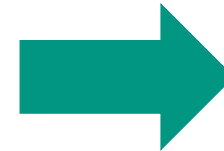
Beispiel



Address	Data		
11	0	1	0
10	1	0	0
01	1	1	0
00	0	1	1

Depth

Width



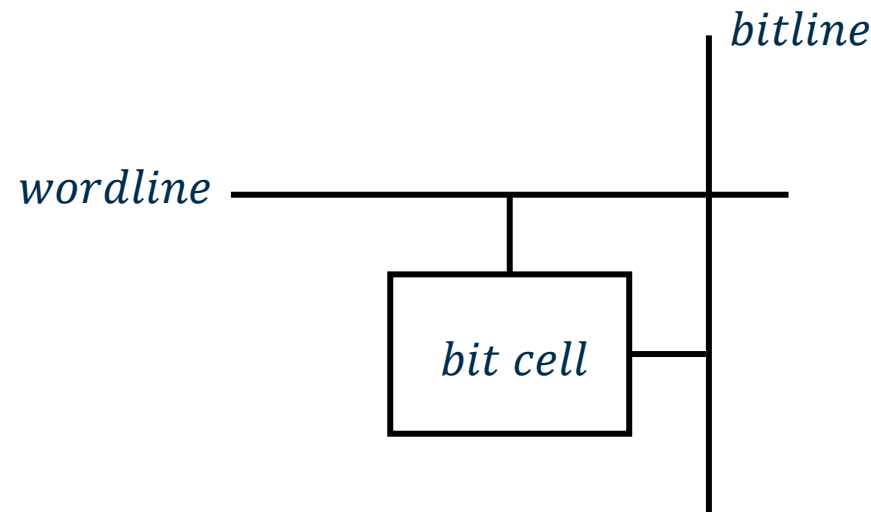
<i>address</i> _{2:0}	<i>data</i> _{3:0}
00	010
01	100
10	110
11	011

S. 264 [1]

Memory Arrays

Bit Cells

- Die Speicherarrays sind als Array von **Bit Cells** aufgebaut. Jede Bitzelle kann ein Bit speichern.
- Jede Zelle ist mit einer **Wordline** und einer **Bitline** verbunden.
- Wenn die Wordline einer Reihe von Bit Cells aktiv ist, senden oder empfangen die entsprechenden Bit Cells ihren Wert über die Bitline.



Referenz S. 264 [1]

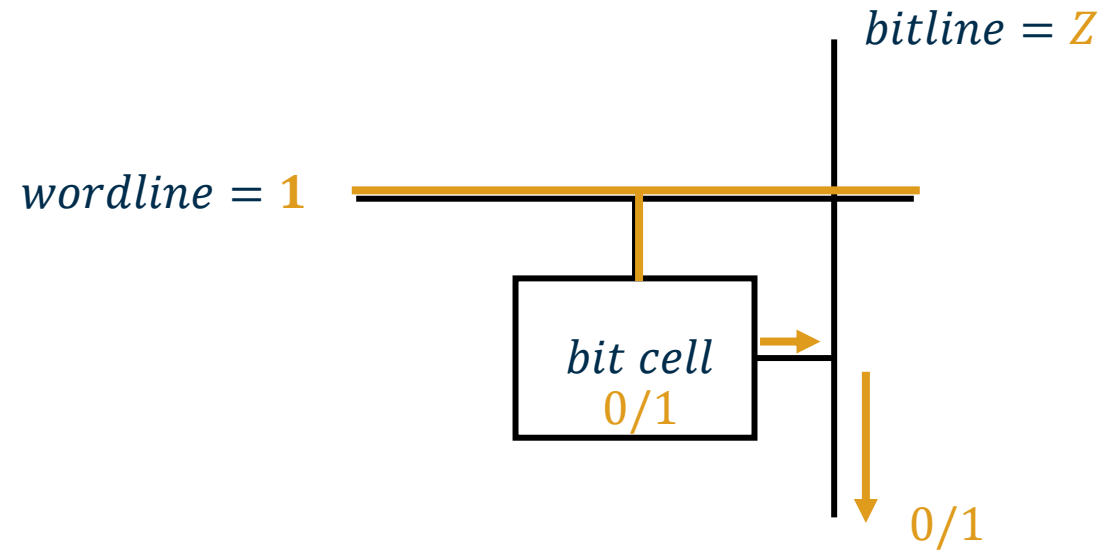
Memory Arrays

Bit Cells

Beispiel

Bitwert lesen.

- Die bitline ist anfangs floating (Z).



Referenz S. 264 [1]

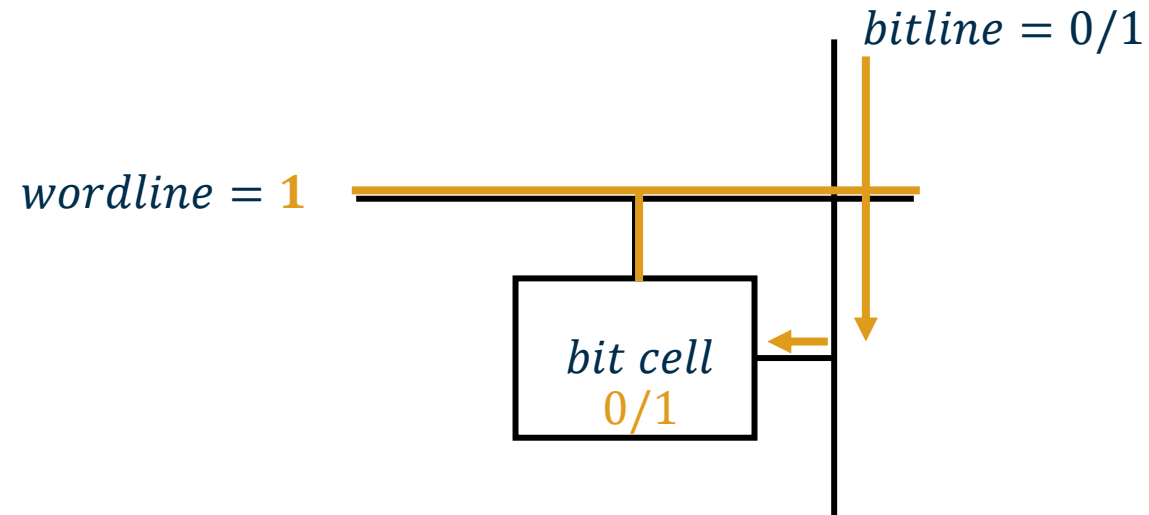
Memory Arrays

Bit Cells

Beispiel

Bitwert schreiben.

- Die bitline wird gesetzt.



Referenz S. 264 [1]

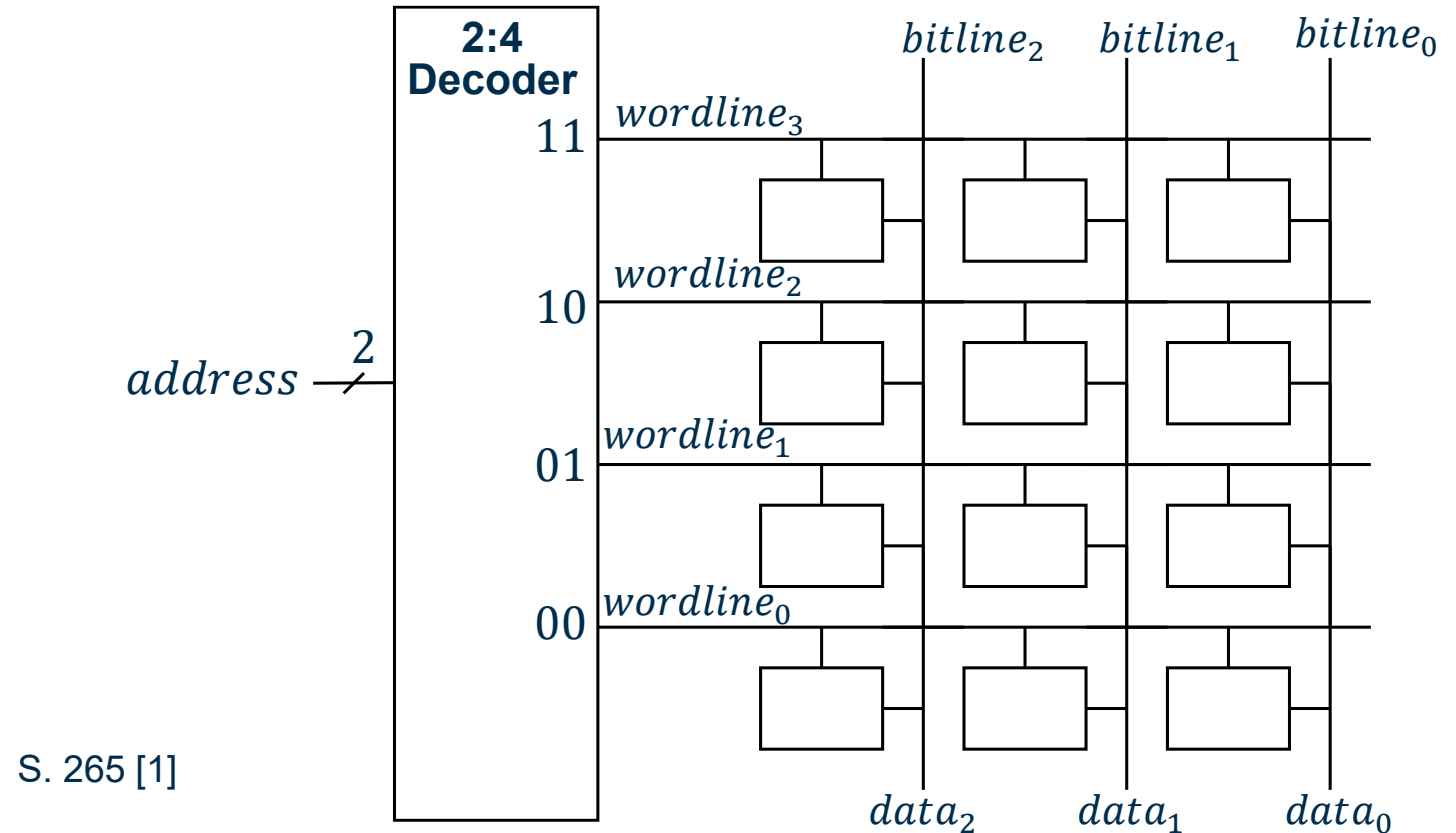
Memory Arrays

Organisation

- Ein Decoder aktiviert die aus dem Adress-Input abgeleitete wordline.

Beispiel

4x3-Speicherarray



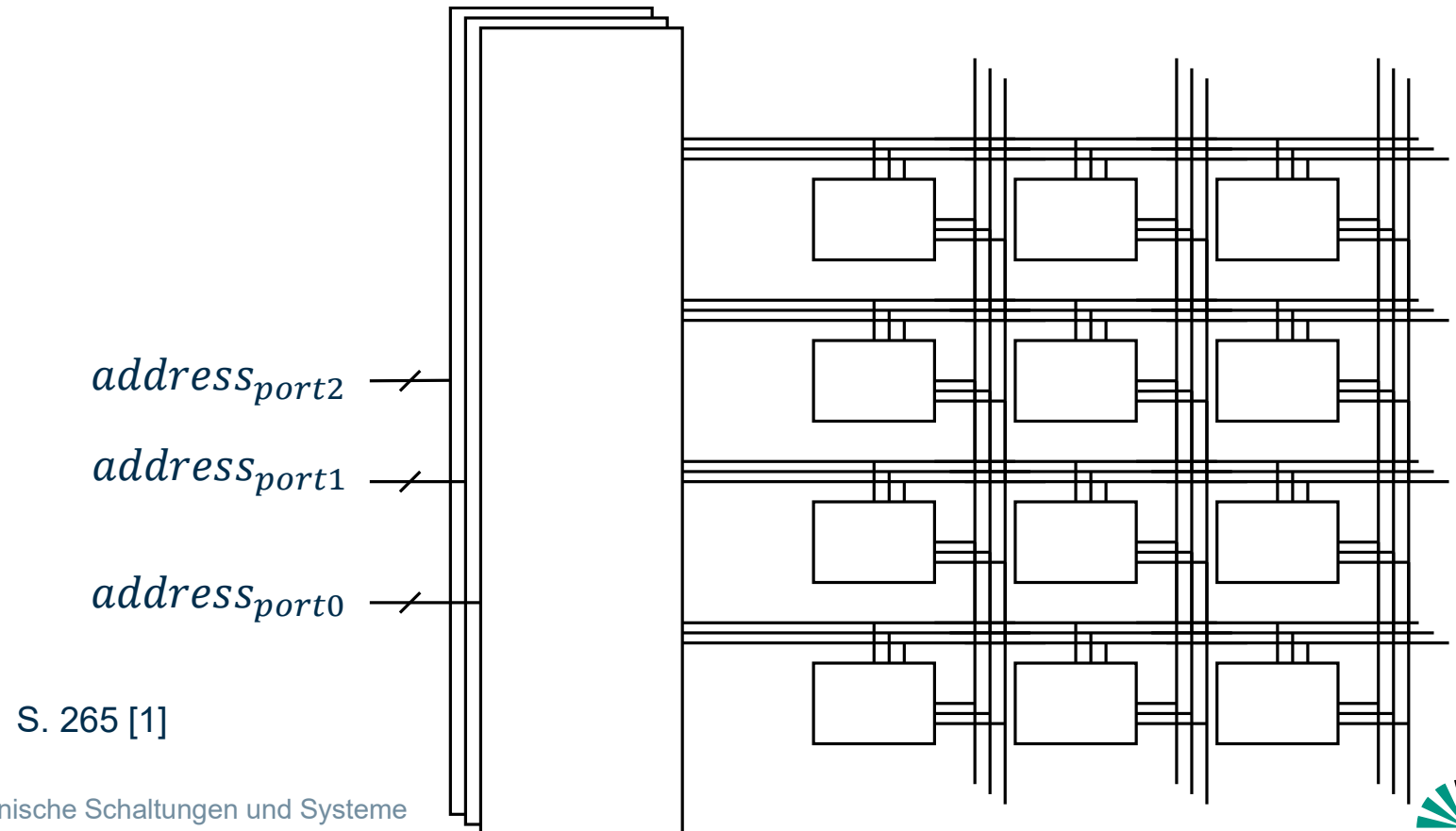
Memory Arrays

Memory Ports

- Der Speicher verfügt über einen oder mehrere **Ports**. Jeder Port gewährt Lese- und/oder Schreibzugriff auf eine Speicheradresse.

Beispiel

3-Port-4x3-Speicherarray

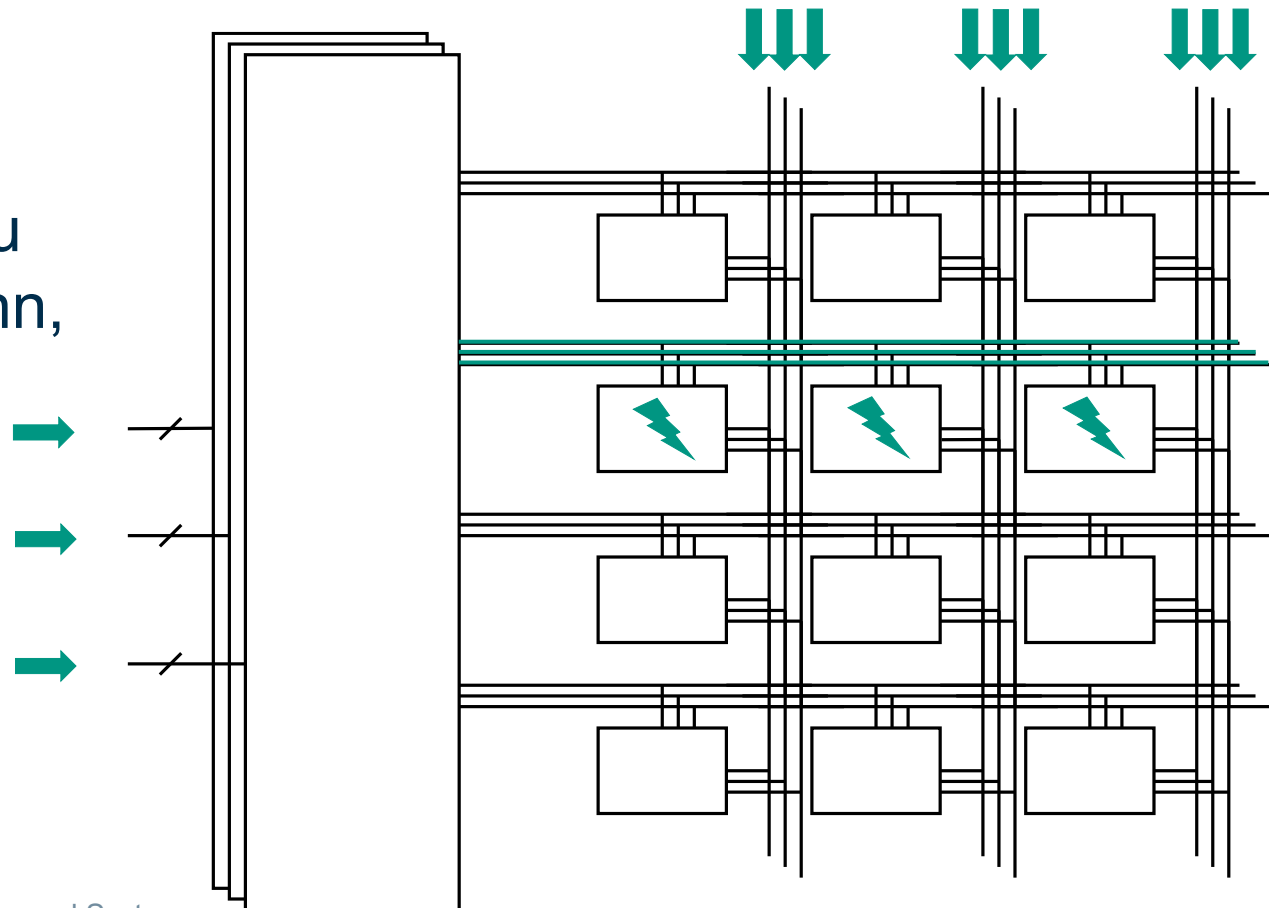


Memory Arrays

Memory Ports

Anmerkung

Während parallele Lesezugriffe auf dieselbe Adresse unkritisch sind, müssen **parallele Schreibzugriff** berücksichtigt werden. Dies führt zu **undefiniertem Verhalten**, es sei denn, das Speichersystem löst solche Konflikte inherent.



S. 265 [1]



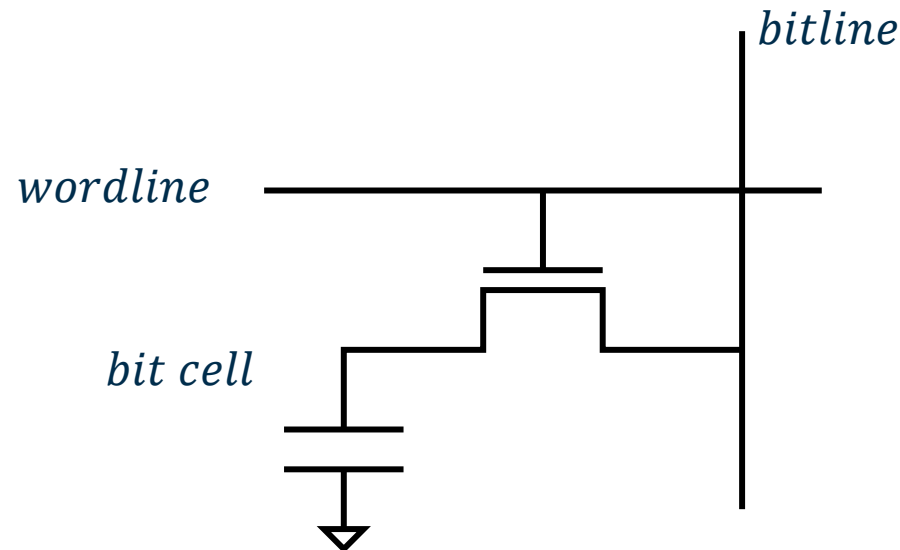
Random Access Memory

2

Random Access Memory

Dynamic RAM

- Bei **dynamic RAM (DRAM)** speichern die Bit Cells ihre Bits als Ladung auf einem **Kondensator**.
- Ein nMOS-Transistor fungiert als **Schalter**, der von der Wordline gesteuert wird und den Kondensator mit der Bitline verbindet oder trennt.
- Die Bit-Zelle ist **dynamisch**, da der Bit-Wert nicht aktiv von einem Transistor getrieben wird.

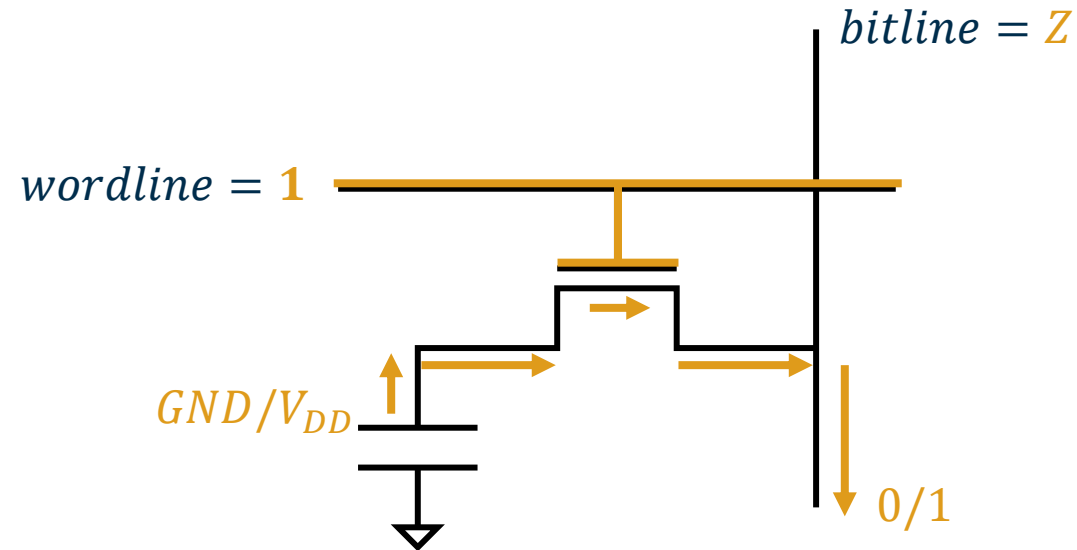


Random Access Memory

Dynamic RAM

Beispiel

Bitwert lesen.



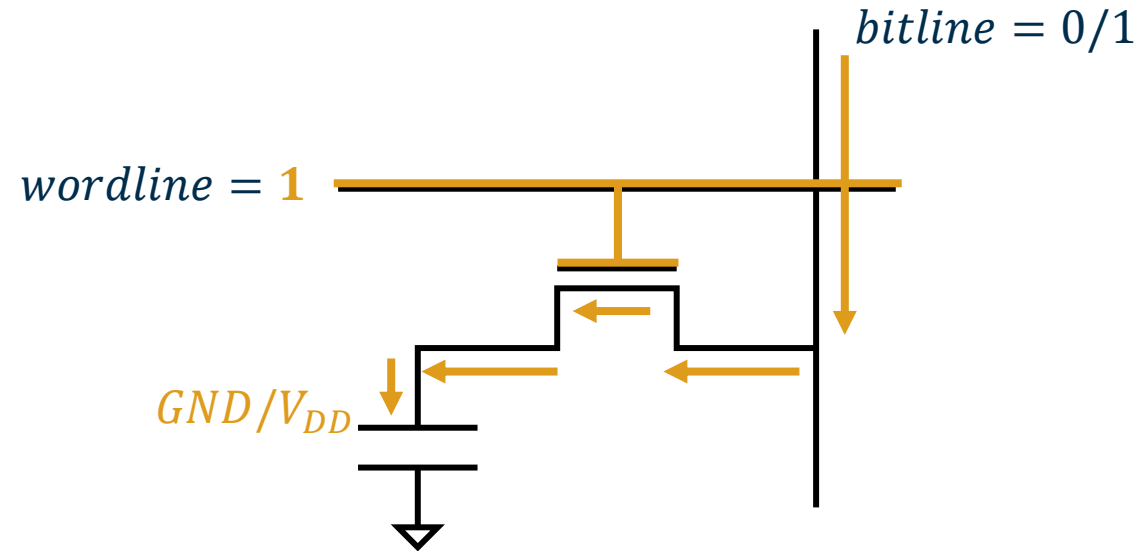
Referenz S. 267 [1]

Random Access Memory

Dynamic RAM

Beispiel

Bitwert schreiben.



Referenz S. 267 [1]

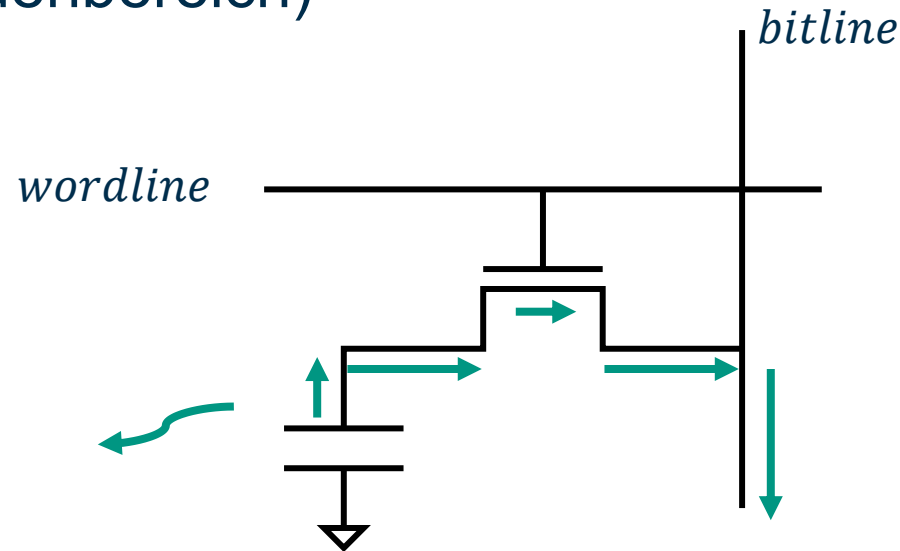
Random Access Memory

Dynamic RAM

Anmerkung

Bei einem Lesezugriff wird der Kondensator **entladen**. Daher muss die Bit Cell nach jedem Lesevorgang erneuert und neu geschrieben werden.

Auch aufgrund von **Leckströmen** muss die Zelle **regelmäßig neu beschrieben werden**. (im Millisekundenbereich)

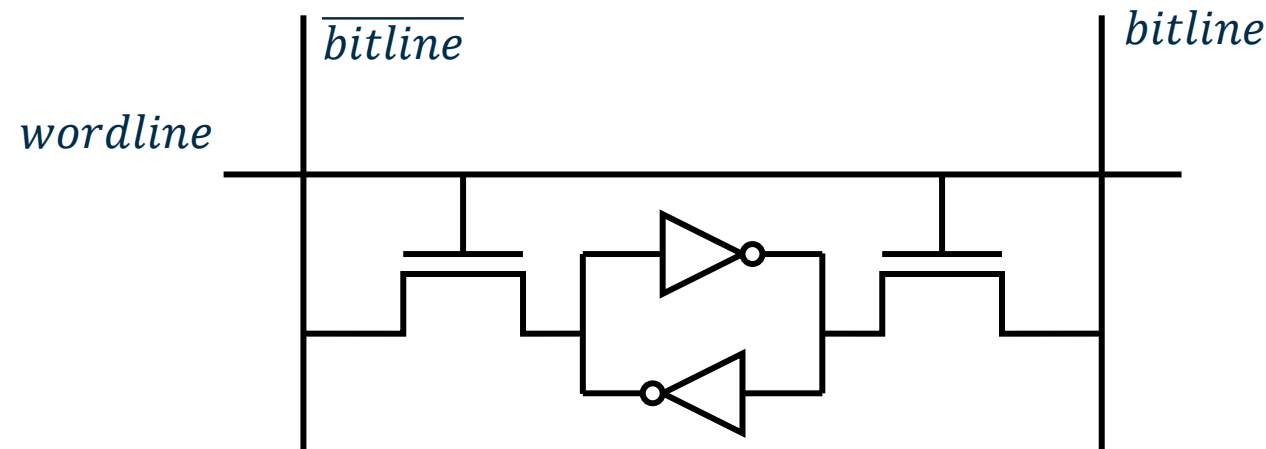


S. 267 [1]

Random Access Memory

Static RAM

- Bei **static RAM (SRAM)** speichern die bit cells ihre Bits mit kreuzgekoppelten (cross-coupled) Invertern.
 - Siehe hierzu auch das Thema „Latches“.
- Die Bit-Zelle ist **statisch**, da der Bit-Wert aktiv getrieben wird und nicht aktualisiert werden muss.
- Beachten Sie, dass die Bit-Zelle über zwei Daten Ein-/Outputleitungen verfügt (*bitline*, *bitline*)



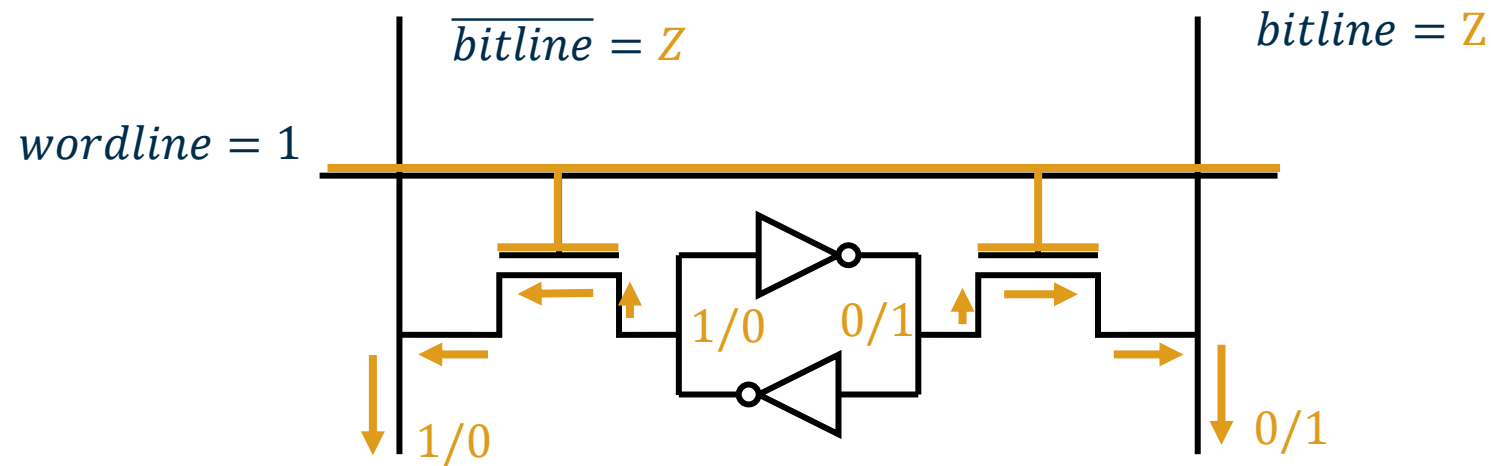
S. 267 [1]

Random Access Memory

Static RAM

Beispiel

Bitwert lesen.



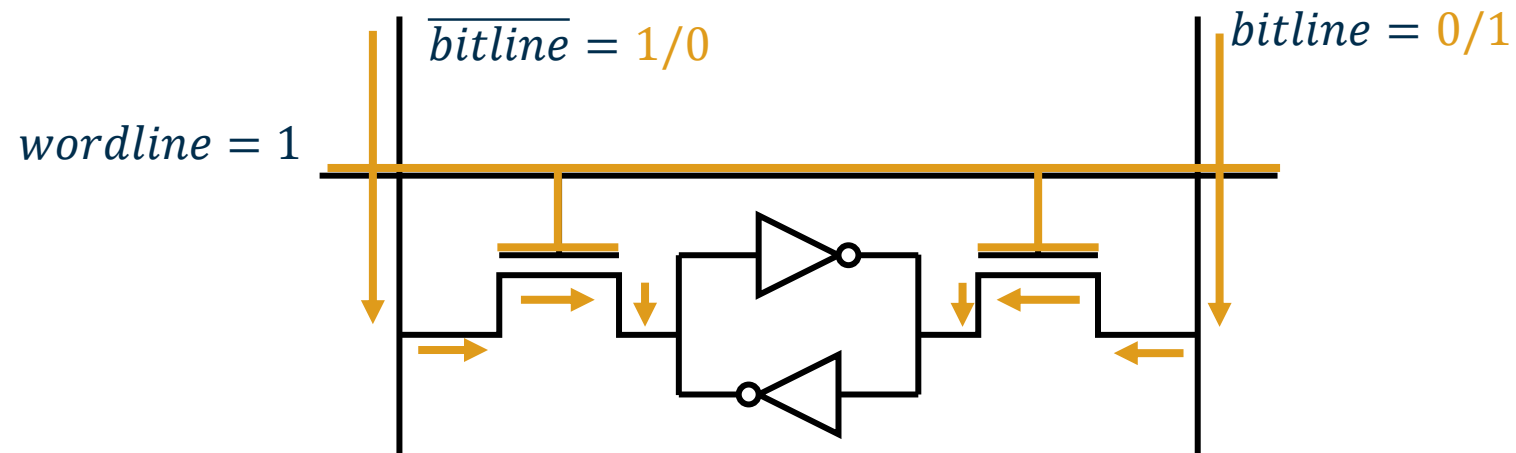
Referenz S. 267 [1]

Random Access Memory

Static RAM

Beispiel

Bitwert schreiben.



Referenz S. 267 [1]

Random Access Memory

Area und Delay (Volatile Memory)

- Die vorgestellten flüchtigen Speicher unterscheiden sich hinsichtlich Flächenverbrauch und Delay Eigenschaften.

Speichertyp	Transistoren pro Bit-Zelle	Latenz
Flip-Flop	~20	schnell
SRAM	6	mittel
DRAM	1	langsam

- Bei **Flipflops** sind die Daten sofort am Output verfügbar.
- Die Latenz von DRAMs ist länger als die von SRAMs, da die Bitleitung nicht aktiv getrieben wird.

Random Access Memory

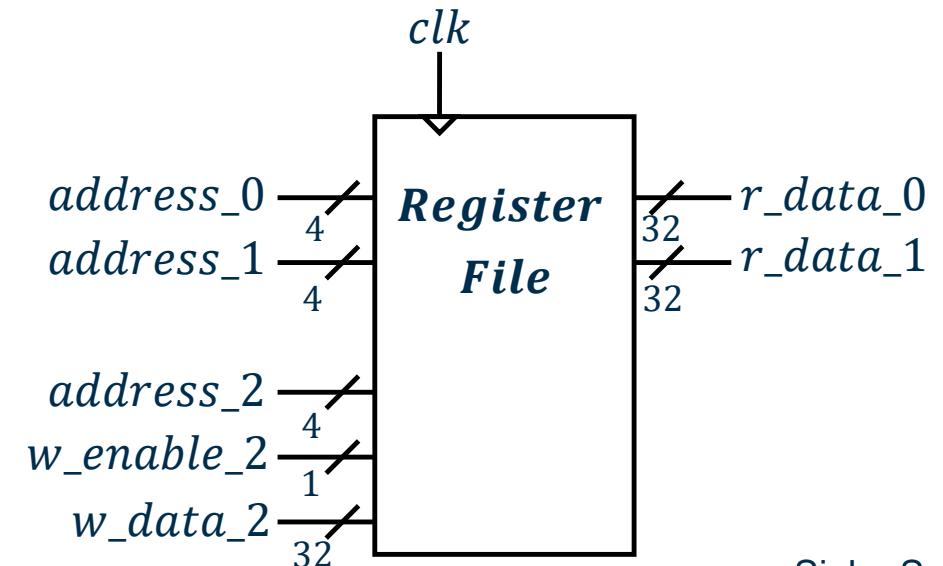
Register Files

- Eine Gruppe von Registern, sogenannte **Register files**, wird häufig in digitalen Systemen zum Speichern temporärer Variablen verwendet.
- Sie wird mit einem **Multi-Port SRAM** Array implementiert, da dies effizienter ist als ein Array aus Flipflops.

Beispiel

16x32-Registerdatei mit drei Ports

- Es können zwei Register gleichzeitig gelesen und ein Register gleichzeitig beschrieben werden.



Siehe S. 268 [1]



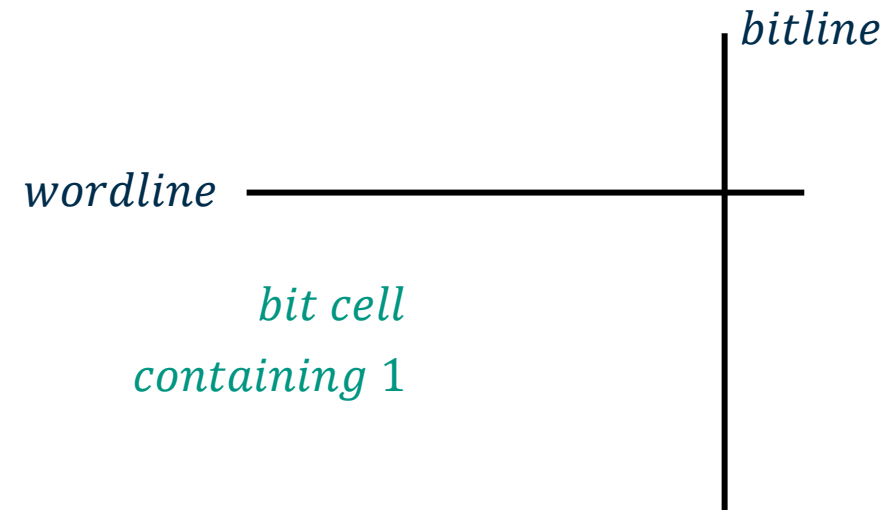
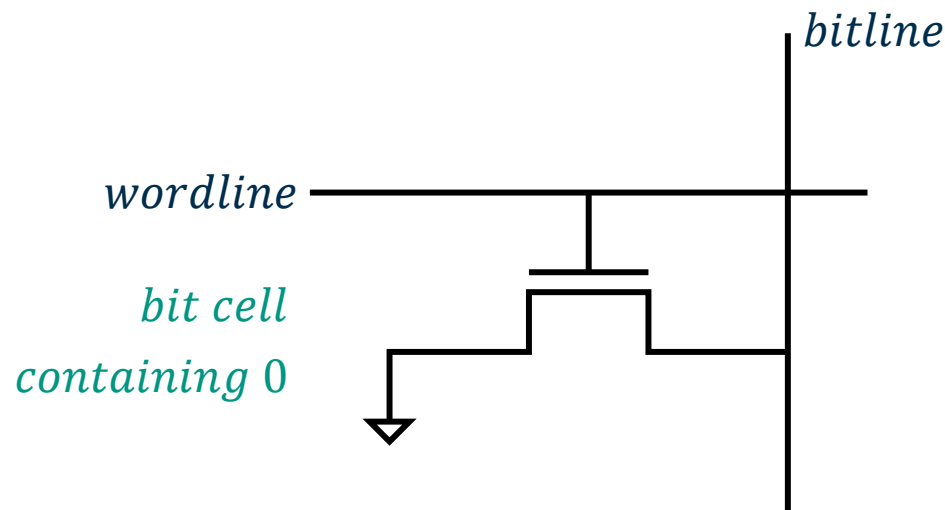
Read Only Memory

3

Read Only Memory

Not-Programmable ROM

- Bei **Read Only Memory (ROM)** werden die bit cells entsprechend dem von ihnen gespeicherten Bitwert implementiert.
- Der ROM speichert das Bit als Vorhandensein oder Fehlen eines **Transistors**. Daher kann die Zelle ihren Wert nicht „vergessen“.



Referenz S. 268 [1]

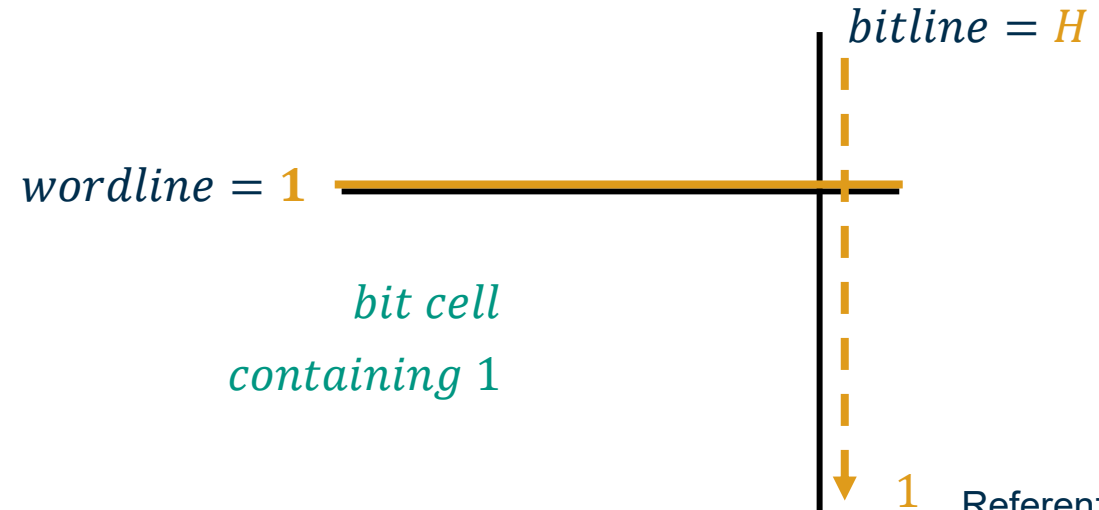
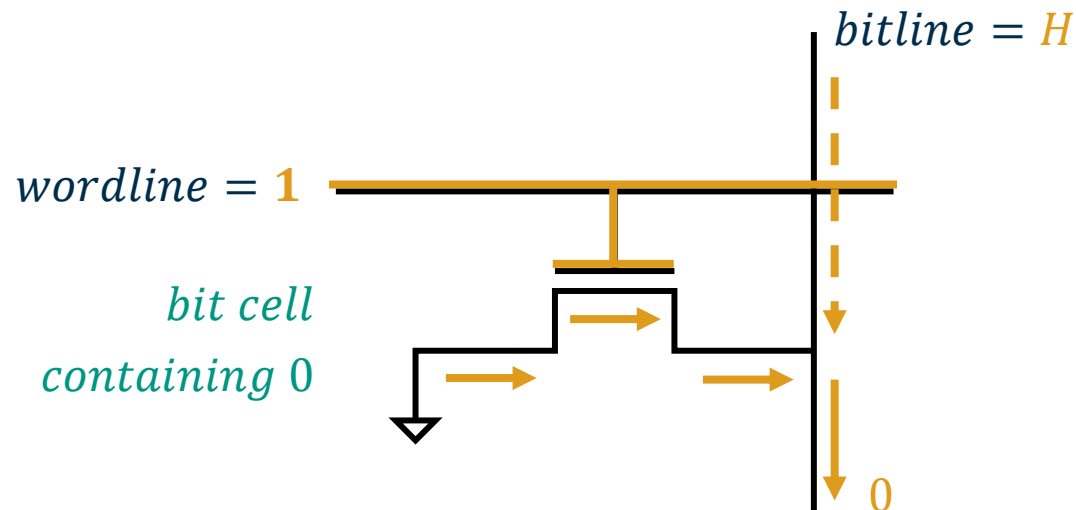
Read Only Memory

Not-Programmable ROM

Beispiel

Bitwert lesen.

- **Anmerkung:** Die bitline ist auf **weak High (H)** gesetzt. Ein weak High auf einer Leitung wird **als 1 gelesen**. Es kann durch eine stärkere Quelle auf 0 oder 1 **überschrieben** werden, ohne dass ein Kurzschluss entsteht.

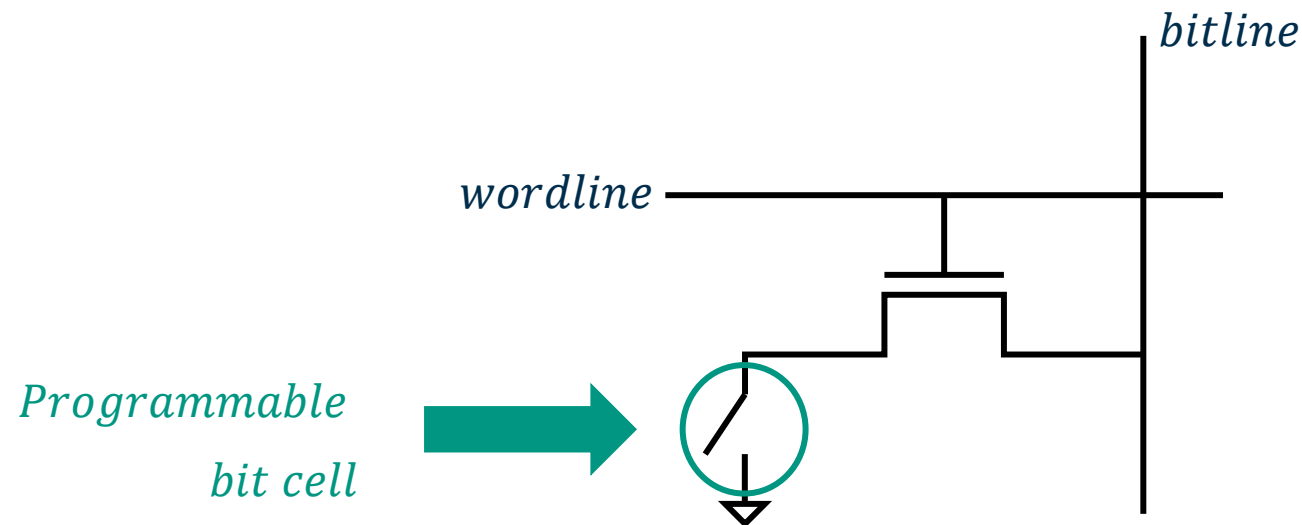


Referenz S. 268 [1]

Read Only Memory

Programmable ROM

- Programmable ROMs (PROM) bieten eine dauerhafte Möglichkeit, GND mit dem Transistor zu verbinden oder zu trennen, der in jeder bit cell platziert ist.



Referenz S. 268 [1]

Read Only Memory

Programmable ROM

Hinweis

- **Reprogrammierbare** ROMs bieten einen reversiblen Mechanismus zum Programmieren der bit cells.
 - **Erasable PROMs (EPROM)** ersetzen den nMOS-Transistor von PROMs durch einen Floating-Gate-Transistor. Dieses Floating Gate kann mit **UV-Licht** gelöscht werden.
 - **Electrical Erasable PROMs (EEPROMs)** und **Flash memory** enthalten Schaltungen zum elektrischen Löschen des Floating-Gate. EEPROM bit cells sind einzeln löschar. Flash-Speicher können nur größere Blöcke löschen, sind aber aufgrund der geringeren Anzahl an Löschsaltungen kostengünstiger.
- ROMs benötigen länger zum Beschreiben, sind aber nonvolatile.

Referenz S. 270 [1]



Lookup Tables

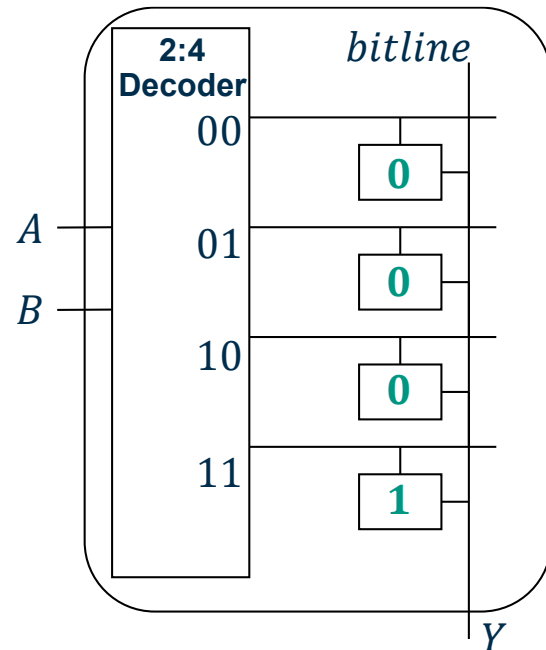
4

Lookup Tables

Logik durch Memory Arrays

- **Memory arrays** werden für logische Funktionen verwendet. (**lookup tables/ LUTs**)
- Die gespeicherten Bits werden je nach Adresswert als Ausgabewerte verwendet. Es kann **jede mögliche Logikfunktion** implementiert werden.

Beispiel: 4-Wort-x-1-Bit-Array für eine two value Logikfunktion



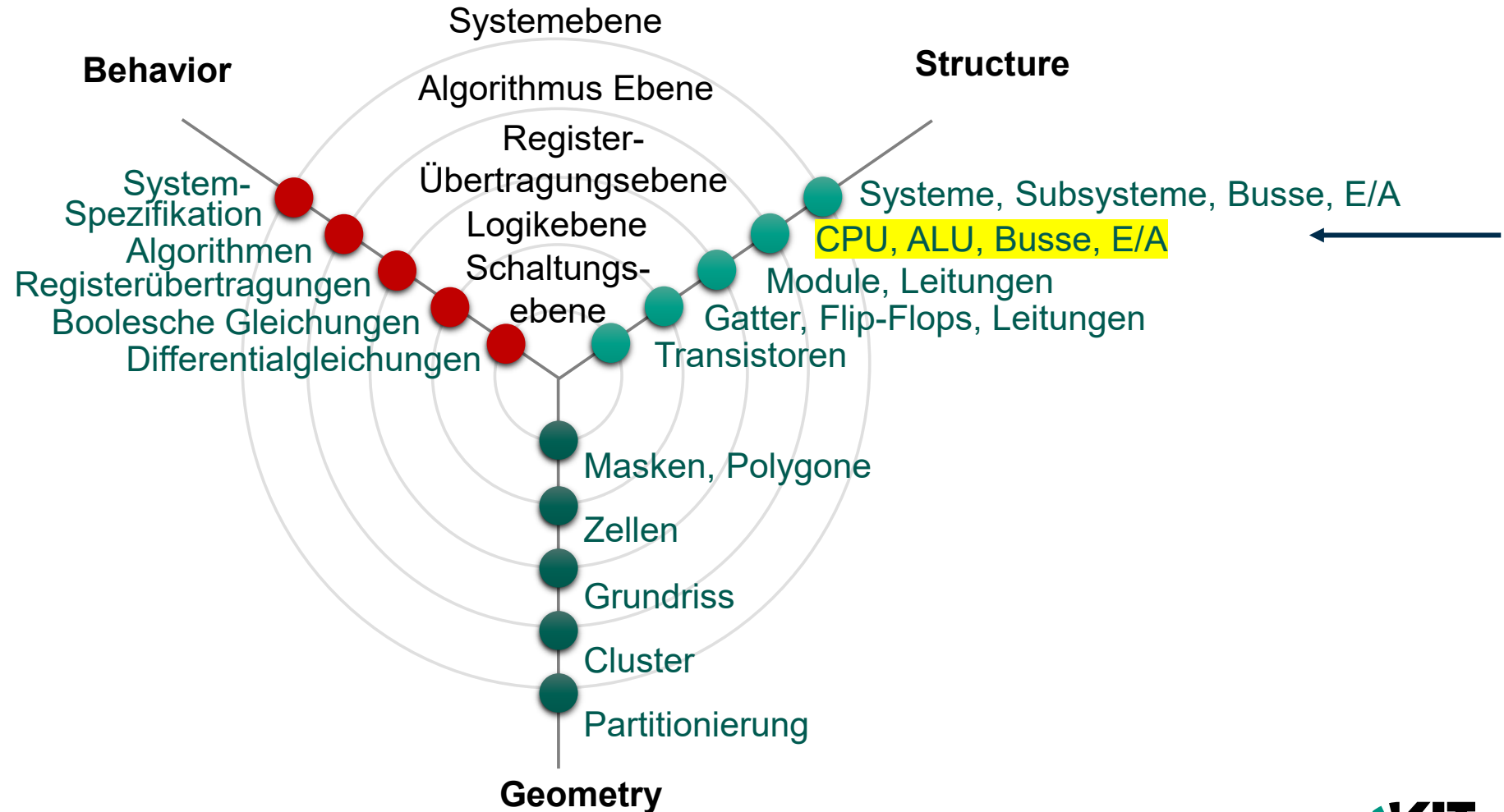
<i>A</i>	<i>B</i>	<i>y</i>
0	0	0
0	1	0
1	0	0
1	1	1

**Two-value
Logic
AND Function**

Siehe S. 270+271 [1]

Advance Organizer

Y-Diagramm



Referenz

- [1] S. L. Harris und D. M. Harris, *Digitales Design und Computerarchitektur*, ARM®-Ausgabe. Waltham, MA: Morgan Kaufmann, 2016.

